SymPy Bug Report

1 Bug 12: Inverse Fourier transform drops Abs in the Lorentzian pair

1.1 Minimal Reproducer

```
from sympy import exp, inverse_fourier_transform, pi, simplify, symbols

t, w = symbols("t w", real=True)
F = 2/(1 + 4*pi**2*w**2)
inv = inverse_fourier_transform(F, w, t)

print("inverse =", inv)
print("at t=-1:", inv.subs(t, -1))
print("expected at t=-1:", exp(-1))
print("difference:", simplify(inv.subs(t, -1) - exp(-1)))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
inverse = exp(-t)
at t=-1: E
expected at t=-1: exp(-1)
difference: 2*sinh(1)
```

SymPy returns e^{-t} as an unconditional inverse Fourier transform. This expression agrees with the correct inverse only on the nonnegative real half line. At $t = -1$, the returned expression evaluates to e, while the expected value is e^{-1}.

1.3 Expected Behavior

The inverse Fourier transform should be

$$\text{inverse\_fourier\_transform}\left(\frac{2}{1+4\pi^2 w^2}, w, t\right) = e^{-|t|}.$$

Equivalently, for real t, the result should be the even function $\exp(-\operatorname{Abs}(t))$, not the one-sided branch $\exp(-t)$.

1.4 Mathematical Explanation

With SymPy's Fourier convention,

$$\mathcal{F}\{f\}(w) = \int_{-\infty}^{\infty} f(t)e^{-2\pi iwt}\,dt, \qquad \mathcal{F}^{-1}\{F\}(t) = \int_{-\infty}^{\infty} F(w)e^{2\pi iwt}\,dw.$$

The standard Lorentzian transform pair is

$$\int_{-\infty}^{\infty} e^{-a|t|}e^{-2\pi iwt}\,dt = \frac{2a}{a^2 + 4\pi^2 w^2} \qquad (a > 0).$$

For $a = 1$, the inverse transform of $2/(1 + 4\pi^2 w^2)$ is therefore $e^{-|t|}$. The expression e^{-t} is not an equivalent representation of $e^{-|t|}$: for $t < 0$,

$$e^{-t} = e^{|t|} \neq e^{-|t|}.$$

The discrepancy is not a missing multiplicative constant or a harmless branch choice. It changes the decay on the negative half line into exponential growth.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant implementation as `sympy/integrals/transforms.py`, specifically `_fourier_transform` around lines 945–961. The public call `inverse_fourier_transform(F, w, t)` constructs an inverse transform object. Its compute method follows the shared Fourier-transform path and then calls `_fourier_transform` with the inverse-Fourier parameters. That helper evaluates the real-axis kernel integral

$$\int_{-\infty}^{\infty} F(w)e^{2\pi iwt}\,dw.$$

For the reproducer, the raw integral result is a `Piecewise` value whose first branch is a closed form valid only under the condition $t > 0$, with an unevaluated integral as the fallback branch. The diagnosed problematic rule is that `_fourier_transform` accepts a `Piecewise` integral result by taking only the first branch and returning that branch expression together with its condition:

```
if F.is_Piecewise:
    F, cond = F.args[0]
    return _simplify(F), cond
```

In this case, the first branch simplifies to $\exp(-t)$ and its condition is $t > 0$. The public Fourier transform wrapper uses `noconds=True` by default, so the condition is discarded before the result is returned to the caller. The source-level failure is therefore a conditional branch being promoted to an unconditional transform result. A plausible fix direction, supported by the diagnosis, is to avoid collapsing a conditional `Piecewise` transform integral to its first branch unless that condition is globally valid; otherwise SymPy should preserve the conditional result, compute the missing branch, or leave the transform unevaluated when `noconds=True` would hide a necessary condition.

1.6 Additional Failing Instances

The same error appears for the parametrized Lorentzian family

$$F_a(w) = \frac{2a}{a^2 + 4\pi^2 w^2}, \qquad a > 0.$$

Mathematically, this is the Fourier transform of $e^{-a|t|}$, so every inverse transform in the family should be even in t. The verification file checks $a = 1, 2, 3, 4, 5, 6$ at $t = -1$; in each case the returned positive-t branch gives e^{a} rather than e^{-a}.

Input / Expression	SymPy behavior	Expected behavior
$2/(1+4\pi^2w^2)$	returns e^{-t}; at $t=-1$, e	$e^{-\lvert t\rvert}$; at $t=-1$, e^{-1}
$4/(4+4\pi^2w^2)$	returns e^{-2t}; at $t=-1$, e^2	$e^{-2\lvert t\rvert}$; at $t=-1$, e^{-2}
$6/(9+4\pi^2w^2)$	returns e^{-3t}; at $t=-1$, e^3	$e^{-3\lvert t\rvert}$; at $t=-1$, e^{-3}
$8/(16+4\pi^2w^2)$	returns e^{-4t}; at $t=-1$, e^4	$e^{-4\lvert t\rvert}$; at $t=-1$, e^{-4}
$10/(25+4\pi^2w^2)$	returns e^{-5t}; at $t=-1$, e^5	$e^{-5\lvert t\rvert}$; at $t=-1$, e^{-5}
$12/(36+4\pi^2w^2)$	returns e^{-6t}; at $t=-1$, e^6	$e^{-6\lvert t\rvert}$; at $t=-1$, e^{-6}

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and found no public issue, pull request, release-note entry, Stack Overflow post, or mailing-list thread describing this *specific* Lorentzian inverse transform returning $\exp(-t)$ instead of $\exp(-\operatorname{Abs}(t))$. The underlying failure family, however, is already documented in the same `_fourier_transform` code path. SymPy issue #22787 (*Bug in fourier_transform*, open, still reproducing on 1.14.0) reports `fourier_transform`/`_fourier_trans` returning a non-even result for a real even input, using the same evenness argument, and issue #14624 (closed) reports `inverse_fourier_transform` giving a wrong answer on the negative half-line. The deduplication verdict is therefore *family known, specific instance new*: this is a new concrete instance of the known, still-open `_fourier_transform` Piecewise-branch failure family, not a novel bug. The case remains individually actionable because it gives a concrete, reproducible transform-pair counterexample and a narrow regression test, and the regression test may be linked to issue #22787.

1.8 Proposed SymPy Regression Test

```python
import pytest
from sympy import Abs, exp, inverse_fourier_transform, pi, simplify, symbols


@pytest.mark.parametrize("a", [1, 2, 3, 4, 5, 6])
def test_inverse_fourier_lorentzian_is_even(a):
    t, w = symbols("t w", real=True)
    expr = 2*a/(a**2 + 4*pi**2*w**2)
    inv = inverse_fourier_transform(expr, w, t)
    assert simplify(inv.subs(t, -1) - exp(-a*Abs(-1))) == 0
```